

פרוייקט גמר בקורס "מבנה המחשב" / תשפ"ו סמסטר א'

מגשים:

עומר טורס ת.ז. – 208004903

רועי הראל ת.ז. – 212358691

טל אבן זהב ת.ז. – 209190321

מרצה הקורס: שי אבידן

חלק א' - תיאור הפרוייקט

מטרת הפרוייקט היא לדמות את אופן הפעולה של מעבד SIMP בעזרת תכנות בשפת C.

הפרוייקט מחולק לשני חלקים עיקריים. הראשון הוא האסמבלר, שתפקידו לקרוא קבצי קלט בשפת אסמבלי, לתרגם אותם לשפת מכונה ולייצר תמונת זיכרון התחלתית עבור המעבד. השני הוא הסימולטור, אשר מסמלץ את מעבד ה-SIMP, מקבל את מוצא האסמבלר (תמונת זיכרון התחלתית המכילה הוראות לביצוע בשפת מכונה) יחד עם קבצים נוספים, ומבצע את ההוראות לפי הסדר הנדרש. בנוסף, כתבנו קבצי קלט בשפת אסמבלי הבודקים את פעולת המעבד.

הפרוייקט מומש באמצעות שתי תכניות נפרדות בסביבת Visual Studio Community edition 2022, אחת לכל חלק בפרוייקט.

עבור כל חלק הוגדרו שני קבצים; הראשון, קובץ header המכיל קבועים, מבני הנתונים והגדרות של המשתנים והפונקציות השונות שנעשה בהן שימוש. השני, קובץ source בו מומשה פונקציית main וכל שאר הפונקציות. הרחבה על מטרתן ואופן פעולתן של הפונקציות מופיעה בפירוט הרלוונטי לכל תוכנה.

המעבד מכיל מערך זיכרון של 16 רגיסטרים פנימיים ברוחב 32 ביטים, כאשר ייעוד כל רגיסטר נקבע בהגדרת הפרוייקט. בנוסף למערך הרגיסטרים הפנימיים, המעבד תומך במערך בעל 23 רגיסטרי חומרה שבעזרתם ניתן להתממשק עם התקני קלט/פלט: המעבד תומך בקלט\פלט לפי ההוראות שהוא מקבל (על ידי הוראות מקודדות שמגיעות מהאסמבלר) ומכיל מספר רכיבים כדי לתמוך בפונקציונליות הנדרשת. המעבד מחזיק בנוסף בזיכרון ראשי ברוחב 20 ביט ובעומק של 4096 שורות. אל המעבד מחובר דיסק קשיח המורכב מ-128 סקטורים, ובכל סקטור יש 128 שורות ברוחב 20 ביט גם כן. בנוסף לדיסק הקשיח יש עוד מספר התקני קלט/פלט – נורות led, צג 7segment ומוניטור מונוכרומטי.

חלק ב' – האסמבלר

תיאור כללי

תוכנית האסמבלר אחראית על תרגום קוד המקור (קובץ asm) לשפת מכונה. הפלט הוא קובץ טקסט (memin.txt) המייצג את תמונת הזיכרון של המחשב, אשר ישמש כקלט לסימולטור. התוכנית נכתבה בשפת C ומממשת אלגוריתם של **שני מעברים** (Two-Pass Assembler), על מנת לאפשר תמיכה בקפיצות לתוויות (Labels) אשר מוגדרות בהמשך הקוד.

האסמבלר הוא אפליקציית Command Line המורצת באופן הבא:

```
asm.exe program.asm memin.txt
```

קבצי קלט ופלט

- **קלט** – program.asm : התוכנית הכתובה בשפת אסמבלי.
- **פלט** – memin.txt : תמונת זיכרון התחלתית בשפת מכונה.

מבני נתונים

התוכנית עושה שימוש במבני הנתונים הבאים לצורך ניהול הזיכרון, מיפוי התוויות ותרגום הפקודות:

1. symbol_table

- **תיאור** : מערך של Structs מסוג Symbol כל איבר במערך מייצג לייבל שזוהה בקוד.
- **מבנה האיבר**:
 - char name[51]: מחרוזת לשמירת שם התווית (עד 50 תווים + תו מסיים).
 - int address: מספר שלם המייצג את כתובת הזיכרון (PC) שאליה מצביעה התווית.
- **גודל** : מוגדר ע"י הקבוע MAX_LABELS (500 איברים) .
- **שימוש** : המערך מתמלא במהלך המעבר הראשון (First Pass) ומשמש לפענוח כתובות לקפיצה במהלך המעבר השני.

2. symbol_count

- **תיאור** : משתנה מסוג (int).
- **שימוש** : שומר את מספר הלייבלים הנוכחי שנמצאו והוכנסו לטבלה. המשתנה מאותחל ל-0, ובכל פעם שנוסף לייבל חדש הוא משמש כאינדקס לכתיבה במערך ומקודם ב-1.

3. תמונת הזיכרון (memory)

- **תיאור** : מערך גלובלי של מספרים שלמים (int).
- **גודל** : מוגדר ע"י הקבוע MEMORY_SIZE (4096 מילים, תואם לגודל הזיכרון של המעבד).
- **שימוש** : המערך משמש כבאפר לשמירת קוד המכונה במהלך העיבוד. בסיום הריצה, תוכן המערך נכתב לקובץ הפלט. האינדקס במערך מייצג את הכתובת בזיכרון.

4. Lookup Tables

- **Opcodes**: מערך מחרוזות קבוע (const char*) המכיל את שמות כל הפקודות הנתמכות (כגון add, beq, halt). האינדקס של המחרוזת במערך תואם לערך ה-Opcodes המספרי שלה.
- **Registers**: מערך מחרוזות קבוע המכיל את שמות הרגיסטרים (כגון \$t0, \$sp, \$imm). המערך משמש להמרה מהירה בין שם הרגיסטר באסמבלי למזהה המספרי שלו (0-15).

פונקציות

פונקציות ראשיות

- **main(int argc, char *argv[])** הפונקציה הראשית. אחראית על קבלת שמות קבצי הקלט והפלט משורת הפקודה, פתיחת הקבצים וניהול זרימת התוכנית (קריאה ל-first_pass ואחריה ל-second_pass). בסיום, היא כותבת את תוכן מערך ה-memory לקובץ הפלט memin.txt וסוגרת את הקבצים.
- **first_pass(FILE *input)** מבצעת סריקה ראשונית של הקובץ למיפוי תוויות.
- **second_pass(FILE *input)** מבצעת את התרגום הסופי לשפת מכונה.

נדון במעברים בהרחבה בהמשך.

פונקציות עיבוד וניתוח (Parsing)

- **parse_line(char *line, char *opcode, char *rd, char *rs, char *rt, char *imm)** מפרקת שורת פקודה למרכיביה.
- **parse_int(char *str)** ממירה מחרוזת למספר שלם. הפונקציה תומכת בשני פורמטים:

1. מספר עשרוני (למשל 100)
2. מספר הקסדצימלי (למשל 0x100 מתבצע זיהוי של הקידומת 0x)

פונקציות עזר ומיפוי

- **get_opcode_id(char *name)** סורקת את מערך ה-opcodes-הגלובלי ומחזירה את האינדקס (0-21) התואם לשם הפקודה. מחזירה 1-אם הפקודה אינה מוכרת.
- **get_register_id(char *name)** סורקת את מערך ה-registers-הגלובלי ומחזירה את האינדקס (0-15) התואם לשם הרגיסטר. מחזירה 1-אם השם אינו תקין.
- **trim_whitespace(char *str)** פונקציית שירות המשנה את המחרוזת in place. היא מסירה רווחים מתחילת המחרוזת ומסופה כדי למנוע שגיאות בעיבוד.

ניהול טבלת הסמלים

- **add_symbol(char *label, int address)** מוסיפה לייבל חדש למערך symbol_table. הפונקציה שומרת את שם הלייבל ואת הכתובת (PC) שבה הוא הוגדר, ומקדמת את המונה symbol_count.

- **get_symbol_address(char *label)** מחפשת שם של לייבל ב symbol_table ומחזירה את הכתובת המשויכת אליו. משמשת במעבר השני להחלפת שם הלייבל בכתובת מספרית.

מימוש ותהליך הריצה

1. מעבר ראשון – טיפול בלייבלים בשלב הראשון, התוכנית סורקת את קובץ הקלט מתחילתו ועד סופו במטרה לבנות את Symbols Table

- **זיהוי לייבל:** הסורק בודק בכל שורה האם המילה הראשונה מסתיימת בתו נקודתיים. במידה וכן, שם התווית נשמר ב symbol_table יחד עם כתובת ה PC הנוכחית.
- **חישוב כתובות:** כדי לשמור את הכתובת המדויקת של כל תווית, האסמבלר מחשב את גודל הפקודה הנוכחית (1 או 2 מילים) ומקדם את מונה ה PC בהתאם. בשלב זה עדיין לא מתבצעת כתיבה לזיכרון.
- **2. מעבר שני – לולאת התרגום (Second Pass)** לאחר בניית טבלת הסמלים, התוכנית מבצעת מעבר נוסף על הקובץ לצורך התרגום המלא. הלולאה קוראת את הקובץ שורה אחר שורה ומבצעת את הפעולות הבאות:
 - **פרסור השורה (Parsing):** השורה מפורקת לרכיבים Opcode, rd, rs, rt, imm : הסורק מפריד את המחרוזת לפי רווחים, פסיקים וסוגריים, ומתעלם מהערות. במידה ויש לייבל בתחילת השורה, הוא מדולג (מכיוון שטופל במעבר הראשון).
 - **תרגום Opcodes:** התוכנית משתמשת במערך opcodes כדי למצוא את הקוד המספרי של הפקודה. מידע זה משמש גם לקביעת סוג הפקודה.
 - **תרגום הרגיסטרים:** ארגומנטים המייצגים רגיסטרים מתורגמים למזהה המספרי שלהם (0-15) בעזרת המערך registers. ערכים אלו משובצים במקומות המתאימים במילת הפקודה (Bits 0-11).
 - **טיפול ב Immediate ובלables:** האסמבלר בודק האם הפקודה דורשת שימוש Immediate או בלייבל:
 - אם הארגומנט הוא מספר, הוא מומר לערך בינארי.
 - אם הארגומנט הוא לייבל (למשל בפקודת beq) האסמבלר שולף את כתובת התווית מטבלת הסמלים. במקרים אלו, האסמבלר מזהה שימוש ברגיסטר \$imm ומקודד את הפקודה כך שתתפוס שתי מילות זיכרון (המילה הראשונה לפקודה והשנייה לערך הקבוע).

3. יצירת הפלט בסיום עיבוד כל הפקודות ושמירתן במערך MEMORY, האסמבלר עובר על המערך וכותב את התוכן לקובץ memmin.txt. הכתיבה מתבצעת בפורמט הקודצ'ימלי (5 ספרות, רוחב 20 ביט), כאשר כל שורה בקובץ הטקסט מייצגת כתובת בזיכרון.

חלק ג' - Simulator

תיאור המימוש:

תפקידו של הסימולטור בפרויקט זה הוא לדמות פעילות שוטפת של מעבד SIMP הכוללת:

ביצוע הוראות - בהתאם להגדרת ההוראות בפרויקט, לפי לולאת ה- fetch-decode-execute.
ממשק עם התקני קלט-פלט - כתיבה וקריאה מהדיסק הקשיח, ממשק עם המסך המונוכרומטי, ממשק עם צג ה- 7segment וכן ממשק עם נורות LED
טיפול בפסיקות השונות - פסיקת טיימר (irq0), פסיקת הדיסק (irq1) ופסיקת קו חיצוני (irq2).
תיעוד הפעילות - ביצוע מעקב (trace) ותיעוד אחרי תהליך העבודה וכתיבה לקבצי מעקב אחר מצב הרגיסטרים הפנימיים, רגיסטרי החומרה נורות ה- led וצג ה- 7segments.
בדומה לאסמבלר, הסימולטור מומש בשפת C, בסביבת VS.

קבצי קלט:

- memin.txt - תמונת זיכרון התחלתית, פלט של האסמבלר.
- diskin.txt - תמונת זיכרון התחלתית בדיסק הקשיח.
- irq2in.txt - רשימת מחזורי השעון בהם תתרחש פסיקה מספר 2

קבצי פלט:

- memout.txt - תמונת זיכרון **לאחר** הרצת הסימולטור.
- regout.txt - תמונת הרגיסטרים הפנימיים **לאחר** הרצת הסימולטור.
- cycles.txt - כמות מחזורי השעון שעברו **במהלך** הרצת הסימולטור.
- diskout.txt - תמונת הזיכרון בדיסק הקשיח **לאחר** הרצת הסימולטור.
- monitor.txt - קובץ מכיל שורות הקסדצימליות אשר מציגות תמונה שמוצגת על גבי המסך.
- monitor.yuv - קובץ מכיל שורות **בינאריות** אשר מציגות תמונה על גבי במסך **לאחר** ריצת סימולטור.
- trace.txt - מעקב אחרי ההוראות, הכתובת שלהן ותמונת הרגיסטרים הפנימיים **במהלך** ריצת הסימולטור.
- hwregtrace.txt - מעקב אחרי כתיבה או קריאה מרגיסטרי החומרה **במהלך** ריצת הסימולטור.
- leds.txt - קובץ המכיל שינויים במערך ה- leds **במהלך** ריצת הסימולטור.
- display7seg.txt - קובץ המכיל שינויים (שינוי בתצוגת הצג) בצג 7segment **במהלך** ריצת הסימולטור.

מבנה הנתונים instruction

מדמה הוראה בודדת בשפת מכונה, מאפשר אחסון חלקים נפרדים של הוראה במבנה אחד לטובת גישה נוחה ומהירה יותר. שדות:

Unsigned int opcode - 8 bits, unsigned int rd – 4 bits, unsigned int rs - 4 bits, unsigned int rt -4 bits, int imm – 20 bits, int is_itype.

משתנים גלובליים

תיאור	Gloal variable
מערך רגיסטרי ה-PC	regs[NUM_REGS]
מערך רגיסטרי החומרה	io_regs[NUM_IO_REGS]
מערך הזכרון הראשי	mainMem[MEM_SIZE]
מערך הזכרון בדיסק	hard_disk[DISK_SIZE]
מערך המוניטור	monitor[MONITOR_SIZE]
מערך דינמי של כל הכתובות בהן יש פסיקה.	*irq2_cycle
אינדקס לקריאת הכתובות שבהן יש פסיקה למערך	irq2_count
אינדקס לקריאת הכתובות מהמערך, לבדיקת פסיקה במחזור שעון.	irq2_index
	branch_condition
	interrupt_flag
	clock_cycle
בהוראת halt מסיים את ריצת הסימולטור	halt_condition
נדלק כאשר ניתנת פקודת קריאה/ כתיבה והיא מתבצעת	disk_busy
טיימר לפעולת הדיסק, לבקרת זמן הטיפול	disk_timer
מצב רגיסטר 14- כתיבה, קריאה או no command	disk_cmd_type
מספר הסקטור לקריאה או כתיבה	disk_sector_num

כתובת לקריאה או כתיבה	disk_buffer_addr
מצב leds אחרון	last_leds_value
מצב צג 7segments אחרון	last_display_value

פונקציות

תיאור	Functions
	Setup Functions
מאתחלת במערך את הזכרון הראשי מקובץ הקלט המתאים.	void FillMainMem(FILE *pmemin)
מאתחלת במערך את זכרון הדיסק מקובץ הקלט המתאים..	void FillDiskinArr(FILE *pdiskin)
מאתחלת מערך דינמי של כל הכתובות בהן יש פסיקה מקובץ הקלט המתאים.	void FillIrq2inArr(FILE *pirq2in)
טוענת מידע מקבצי הקלט למערכים.	void SetArrays(FILE *pdiskin, FILE *pmemin, FILE *pirq2in)
מרחיבה מ-20 ל-32 ביטים	int signExtension(int val)
	Main process functions
פונקציה שמבצעת בדיקה שוטפת של סטטוס הפסיקות בסימולטור ובמידת הצורך מבצעת אתחול רגיסטרים רלוונטיים לטיפול בפסיקה.	int CheckInterrupts()
פונקציה אשר בודקת האם הדיסק פנוי, ובמידה ולא מתבצע טיפול בפסיקה מספר 1. לאחר תום הזמן (1024 מחזורי שעון) מתבצעת הוראת הקריאה או הכתיבה	void HandleDiskOperation()

הנדרשת, מאותחלים הרגיסטרים הרלוונטיים, ונדלקת אינדיקציה לפסיקה 1.	
קריאת שורה מהזכרון והמרת הקידוד לשדות מבנה הנתונים instruction	Instruction Fetch()
מעבר על ההוראה וביצועה תוך כדי כתיבה לקבצים הנדרשים.	void Execute/Instruction *inst_ptr)
מקדמת את ה-PC בהתאם לפקודה	void AdvancePC/Instruction *inst)
פונקציה אשר מטפלת בקידום מונה מחזורי השעון בסימולטור, נקראת בסוף כל איטרציה של הלולאה.	void AdvanceClock()
סט של פונקציות לכתיבה לקבצי פלט, כל אחת מיועדת לקובץ נפרד.	// Output functions
	void WriteMemout(FILE *fp)
	void WriteRegout(FILE *fp)
	void WriteTrace(FILE *fp, Instruction *inst)
	void WriteHwRegTrace(const char *operation, int addr, unsigned int value)
	void WriteCycles(FILE *fp)
	void WriteLeds(int cycle, unsigned int value)
	void WriteDisplay7seg(int cycle, unsigned int value)
	void WriteDiskout(FILE *fp)
	void WriteMonitor(FILE *monitor_txt, FILE *monitor_yuv)

בהתאם ללולאת fetch-decode-execute עבודת הסימולטור מתחלקת לכמה שלבים:

1. **הכנת תשתית לעבודת הסימולטור:** פתיחת קבצי קלט, קריאה מהם וסגירתם
פתיחת קבצי הפלט המתעדכנים לכתיבה.
2. **הרצת סימולטור:** באמצעות לולאה אשר עוברת על כלל ההוראות בזיכרון הראשי
באמצעות לולאת while, ומבצעת אותן עד שנתקלת בהוראת Halt. בכל איטרציה של
הלולאה, מתקיימות מספר פעולות:
 3. בדיקת מצב הדיסק ופעולה בהתאם.
 4. בדיקת פסיקות - כתיבה לרגיסטרי status לפסיקות הרלוונטיות.
 5. חילוץ הוראה מתוך זיכרון בכתובת PC, פענוחה וטעינתה לתוך מבנה instruction.
 6. כתיבה תמונת מצב הרגיסטרים הפנימיים לקובץ trace.txt טרם ביצוע הוראה.
 7. ביצוע (execute) הוראה בהתאם ל-opcode שחולץ מהזיכרון. יש לשים לב שחלק
מההוראות משנות את ה-PC הבא בתור.
 8. כתיבה לקבצים hwregtrace, leds, display7seg במידה ומתקיימים תנאים לכתיבה
 9. אתחול רגיסטרי חומרה ושינוי PC הבא במידת הצורך לטובת טיפול בפסיקות.
 10. קידום PC בהתאם לסוג הוראה, קידום מונה השעון.
 11. **כתיבה לקבצי הפלט** הבאים לאחר סיום לולאת הריצה (כלומר לאחר קבלת halt):
Memout.txt ,Regout.txt ,Monitor.txt ,Monitor.yuv ,Cycles.txt, Diskout.txt
הכתיבה מתבצעת ע"י העתקה של ערכים ממערכי זיכרון מתאימים במבנה simulator
לתוך הקובץ.
 12. לאחר יציאה מפונקציית run_simultaor מתבצעת **סגירת כלל הקבצים** ושחרור
הזיכרון שהוקצה בצורה דינמית.

חלק ד' - תכנות בדיקה בשפת Assembly

בפרויקט כתבנו ארבעה קבצי קלט באסמבלי שמטרתם לבדוק את פעולת הסימולטור.

1. sort.asm - תוכנה שממיינת מערך של 16 מספרים באמצעות אלגוריתם bubble sort, באמצעות 2 לולאות מקוננות. הלולאה החיצונית רצה על כל המספרים פעם אחת, ונותנת בכל פעם את נקודת ההתחלה עבור הלולאה הפנימית. הלולאה הפנימית בודקת האם המספר הבא בסדר בזכרון קטן מהמספר הנוכחי, ואם כן מחליפה אותם. כך בכל איטרציה של הלולאה החיצונית אנחנו דואגים למיין המספר המתאים לאינדקס שלה, מסוף המערך. בסיום הריצה נבטיח מערך ממין בסדר עולה.
2. Binom.asm - הבינום מומש באמצעות קריאות רקורסיביות המוגדרות לפי הנוסחה הרקורסיבית:
$$\text{binom}(n, k) = \text{binom}(n-1, k-1) + \text{binom}(n-1, k)$$

כך שתנאי העצירה לכל מסלול רקורסיבי הוא: $k=0$ or $n=k$ ואז נוסף 1 לסכום (כלומר התוכנה הגיעה למקרה היציאה).
כאשר הארגומנטים ההתחלתיים נקראים מהזיכרון.
בסוף הריצה הרקורסיבית התוצאה נשמרת בכתובת 0X102 כנדרש.

3. Triangle.asm - תכנית שמציירת משולש ישר זווית מלא בצבע לבן על המסך, כאשר A הינו הקדקוד השמאלי העליון, B הוא התחתון וזוויתו ישרה, ו-C הוא הקדקוד הימני התחתון. צלע AB הינה אנכית וצלע BC אופקית.

האלגוריתם: מתוך הקואורדינטות של שלושת הקודקודים חושבו אורכי הניצבים וערכים קבועים במשוואת הישר. באמצעות שלושת הביטויים האלו בוצע מילוי של המשולש, באמצעות איטרציה על שורת הפיקסלים בניצב האנכי, כאשר לכל שורה מתבצעות איטרציות מילוי הפיקסלים בצורה אופקית תוך בדיקת תנאי חציית היתר לפי משוואת הישר:

$$(Y - Y_A)(X_C - X_A) \leq (Y_C - Y_A)(X - X_A)$$

במידה וזוהתה חריגה מתנאי החצייה מתבצע דילוג לשורה הבאה של הפיקסלים.

4. Disktest.asm - בדיקה זו בוחנת נכונות המימוש של ממשק מול הדיסק הקשיח. במהלך הבדיקה נעשית קריאה של שמונה סקטורים עוקבים מהדיסק (7-0), סכימתם וכתיבת התוצאה לסקטור התשיעי הבא אחריהם.

חלק ה' - סיכום

לסיכום, פרויקט ה-ISA כלל תכנון ויישום של אסמבלר וסימולטור בשפת C המקבלים קבצי קלט בשפת אסמבלי עבור מעבד ה-SIMP ומבצעים את ההוראות כנדרש. במהלך הפרויקט עסקנו בפרטים של ארכיטקטורת המעבד, ארגון הזיכרון, פעולות קלט/פלט וטיפול בפסיקות. על ידי פיתוח סימולטור המדמה את אופן הפעולה של רכיבים אלו, רכשנו תובנות רבות על אופן

הפעולה הפנימי של מעבד מבוסס RISC. האסמבלר אפשר לנו לתרגם קוד אסמבלי באופן מדויק להוראות קריאות בשפת מכונה, מה שאיפשר אינטראקציה חלקה עם מעבד ה-SIMP המסומלץ. לאחר מכן, הסימולטור ביצע את ההוראות הללו ואיפשר לנו לסמלץ את מחזור ה-fetch-decode--execute ולצפות בהתנהגות המעבד בתרחישים שונים כגון מיון מספרים, בעיות מתמטיות וממשק עם התקני קלט פלט.